

Explaining the mechanism of the vulnerability attack "EternalBlue" that spread "WannaCry"

This article provides detailed information about the EternalBlue vulnerability attack to provide technical information that will help you better understand how this vulnerability attack, which has had a major impact on corporations in various industries around the world, works. This article provides detailed information about the EternalBlue vulnerability attack to provide technical information that will help you better understand how this vulnerability attack, which has had a major impact on corporations in various industries around the world, works.

By: William Gamazo

Trend Micro Research June 07, 2017 Reading time: 6 minutes (2750 words)

The vulnerability attack tool "EternalBlue" attracted a lot of attention in the case of the encryption ransomware "[WannaCry](#)," which spread worldwide in mid-May 2017. After "[WannaCry](#)," a series of copycat criminals using EternalBlue appeared one after another, including the fileless ransomware "[UIWIX](#)," the worm "[EternalRocks](#)," and the cryptocurrency-mining bot "[Adylkuzz](#)."

The vulnerability could allow an attacker to execute arbitrary code on a target PC. It was already fixed in March by a Microsoft update published in the [MS17-010](#) security bulletin, and was considered to be of medium to high risk and complexity, similar to [other vulnerabilities disclosed by the Shadow Brokers hacking group](#).

This blog provides a detailed description of the EternalBlue vulnerability to provide more technical information that will help us better understand how this exploit worked, impacting businesses across a wide range of industries around the world.

■ Vulnerability Analysis EternalBlue

is a security flaw when a Windows SMB1.0 (SMBv1) server processes a specific request. More specifically, it is a vulnerability that causes a buffer overflow in the Large Non-PagedPool area when processing File ExtendedAttribute (FEA) by the kernel function `srv!SrvOs2FeaListToNt` in the Windows SMBv1 code. The non-paged pool refers to the memory area used by the kernel, which is the central function of the OS, that cannot be paged out (memory area divided into a certain page size and written to a file). Of these, the area that exceeds the page size is

specifically called the large non-paged pool. The function `srv!SrvOs2FeaListToNt` used here calls the function `srv!SrvOs2FeaListSizeToNt` to calculate the size of the FEA list when converting FEA to NTFEA (Windows NT FEA). In fact, there is a problem here, and an overflow occurs as shown below.

1. The function "`srv!SrvOs2FeaListSizeToNt`" calculates the size of the FEA list and updates the size of the received FEA list.
2. At this time, due to type conversion to an incorrect data type (WORD type, a C language data type that handles 2-byte unsigned integers), the size of the FEA becomes larger than the original value.
3. An incorrect list size causes an overflow on the nonpaged pool when iterating through the conversion of a FEA list to an NTFEA list.

In this article, we analyzed an overflow in the driver `SRV.sys` that handles SMB traffic on Windows 7 OS with build number `6.1.7601.17514_x86`. The vulnerable code is executed in the function `srv!SrvSmbOpen2`. The stack trace is as follows_:

number	Child EBP	RetAddr	function
00	94527bb4	82171149	<code>srv!SrvSmbOpen2</code> → <code>SrvOs2FeaListSizeToNt()</code>
01	94527bc8	821721b8	<code>srv!ExecuteTransaction+0x101</code>
02	94527c00	8213b496	<code>srv!SrvSmbTransactionSecondary+0x2c5</code>
03	94527c28	8214a922	<code>srv!SrvProcessSmb+0x187</code>
04	94527c50	82c5df5e	<code>srv!WorkerThread+0x15c</code>
05	94527c90	82b05219	<code>nt!PspSystemThreadStartup+0x9e</code>
06	00000000	00000000	<code>nt!KiThreadStartup+0x19</code>

Table 1: Stack trace result of the function "`srv!SrvSmbOpen2→SrvOs2FeaListSizeToNt()`"

To analyze the overflow, we placed breakpoints at the following locations:

```
bp srv!SrvSmbOpen2+0x79 ".printf \feasize: %p indatasize: %p fealist
addr:%p\n",edx,ecx,eax,g;"
```

The values of the variables when the program reaches the breakpoint are as follows (values are in hexadecimal with the decimal value shown in parentheses):

- `feasize` (FEA size): `00010000` (65536)

- indatasize (total number of bytes sent): 000103d0 (66512)
- fealist addr: 89e980d8

At this point, the "Total DataCount (total number of bytes of data sent, 66512)" of "NT Trans Request (one of the SMB commands used for sending data, etc.)" is still larger than the size of the FEA list to be sent (65536).

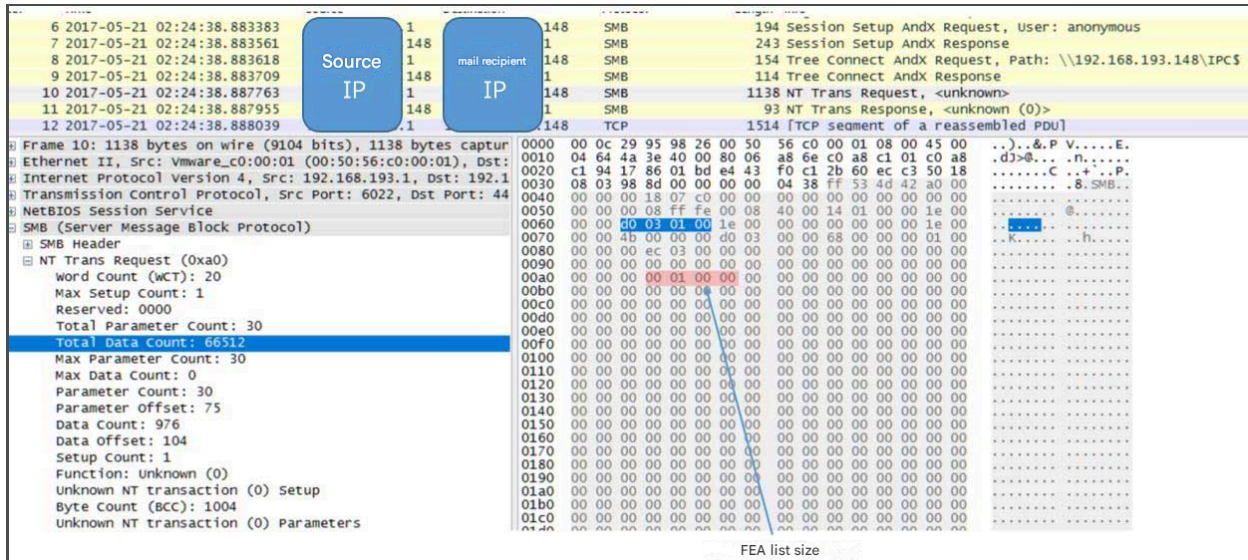


Figure 1: FEA list size (red) and transmitted data size (blue)

The important thing to note here is that the pointer to the transmission data is converted into a FEALIST structure, as shown in Figure 2.

```
typedef struct _FEA { /* fea */
    BYTE fEA; /* flags */
    BYTE cbName; /* name length not including NULL */
    USHORT cbValue; /* value length */
} FEA;

typedef struct _FEALIST { /* feal */
    ULONG cbList; /* total bytes of structure including full list */
    FEA list[1]; /* variable length FEA structures */
} FEALIST, *PFEALIST;
```

Figure 2: FEA and FEALIST structure definition

After converting the send data buffer, the FEA size, which is the value of "FEALIST -> cbList", becomes 00010000 (65536). Next, the memory buffer area required for the SMB driver to convert the FEA list to an NTFEA list is allocated. To allocate this, the function "srv!SrvOs2FeaListSizeToNt" is called to calculate the size of the NTFEA list.

To check the return value, I set a breakpoint as follows:

```
bp srv!SrvOs2FeaListToNt+0x10 ".printf \"feasize before: %p\\n\",poi(edi);r$t0 = @edi;g;"
srv!SrvOs2FeaListToNt+0x15 ".printf \"NTFEA size : %p feasize after:%p\\n\",eax,poi(@$t0);g;"
```

The values of the variables when the program reaches the breakpoint are as follows (values are in hexadecimal with the decimal value shown in parentheses):

- feasize (size of FEA before conversion): 00010000 (65536)
- feasize (size of FEA after conversion): 0001ff5d (130909)
- NTFEA size: 00010fe8 (69608)

As a result, we confirmed that the FEA size, which is the value of FEALIST -> cbList, was updated from 0x10000 (65536) to 0x1ff5d (130909). So where did the inaccurate calculation occur? Figure 3 shows the code that causes the problem.

```
1  unsigned int __stdcall SrvOs2FeaListSizeToNt(PFEALIST FeaList)
2  {
3      _DWORD *v1; // eax@1
4      char *v2; // edi@1
5      char *v3; // esi@1
6      int v4; // ebx@3
7      unsigned int v6; //will be holding the net FEA size to add // [sp+Ch] [bp-4h]@1
8
9      PFEALIST v1 = FeaList; //a1;
10     v6 = 0;
11     v2 = (char *)FeaList + FeaList->cbList; //get pointer to the end of list
12     v3 = FeaList->list; //get pointer to start of the list (char *) (a1 + 1);
13     if (FeaList->list < v2) //the pointer to the start should be before the pointer to the end
14     {
15         while ( v3 + 4 < v2 ) //FEA Header Size has 4 bytes, iterate the complete list.
16         {
17             //v4 = *((_WORD *)v3 + 1) + (unsigned __int8)v3[1];
18             v4 = v3->cbValue + v3->cbName; //get total size of the current FEA
19
20             if ( &v3[v4 + 5] > v2 ) //check if there is room for the next FEA, 5 is because null it is not included in cbName
21                 break;
22
23             if ( RtlSizeTAdd(v6, (v4 + 12) & 0xFFFFF0, &v6) < 0 ) //NTFEA needs a 12 bytes HEADER, increment v6, the NTFEA list size.
24                 return 0;
25
26             v3 += v4 + 5; // move v3 to the next FEA in the list
27             if ( v3 >= v2 ) // There is another FEA?
28                 return v6; // return here, this is not a vulnerable path.
29
30             v1 = a1; //reset v1
31         }
32
33         // we get here when there is room for another FEA but data it not big to store a FEA.
34         *(_WORD *)v1 = (_WORD)v3 - (_WORD)v1; //Let see the values here.
35
36         //here v1 is pointing to FeaList->cbList, so the value is updated.
37         //v3 is pointing to the last invalid FEA
38
39         /*
40         Example:
41         FEAsend: esi = a4217035 = v3
42         FEAsstart: eax = a42070d8 = v1
43
44         sub esi,eax #esi = esi - eax
45         mov word ptr [eax],si
46
47         Result: [eax] = *((_WORD *)v1) 0001ff5d because the cast. the cast is done by:
48         mov word ptr [eax],si
49
50         The spected value should be ff5d
51         */
52     }
53
54     return v6;
55 }
```

Figure 3: Code where the problem occurs when calculating "FEALIST -> cbList"

Lines 40 to 48 in Figure 3 are an example of an inaccurate calculation result. Because the size of the original FEA list has been updated, the repeated process of copying values to the NTFEA list exceeds the size of the NTFEA represented by the return value "v6 (value 00010fe8)". Note that if this function returns a value on line 28 or line 21, the FEA list will not be updated. Note that the variable "v1" will be updated in a case other than the above conditions when using EternalBlue if there is extra buffer space at the end of the FEA list and it is insufficient to store another FEA.

■Kernel memory analysisWhen

a buffer overflow occurs in the large non-paged kernel pool, the following phenomenon is observed in the kernel memory. From the return value of the function "SrvOs2FeaListSizeToNt", the size required to store the NTFEA list is 00010fe8 (69608). Therefore, SRV.sys needs to allocate a kernel pool larger than the page size. In order to accurately trace the events that occur when the FEA list is converted to the NTFEA list, a breakpoint was set as shown below.

```
bp srv!SrvOs2FeaListToNt+0x99 ".printf \"NEXT: FEA: %p NTFEA:%p\\n\\",esi,eax;g;"bp  
srv!SrvOs2FeaToNt+04d ".printf \"MOV2: dst: % p src: %p size:%p\\n\\",ebx,eax,poi(esp+8);g;"
```

```
bp srv!SrvOs2FeaListToNt+0xd5
```

In other words, once the function "SrvOs2FeaListSizeToNt" is called to secure the pool area, this function "SrvOs2FeaToNt" is executed during the loop of converting each element of the FEA list. In this function "SrvOs2FeaToNt", the function "_memmove" is used twice to move memory blocks, and all buffer copy operations are performed using this function. Figure 4 shows the stack trace when converting the FEA list using the breakpoints mentioned above. It is important to note that this process takes a very long time.

```

1  1: kd> g
2  feaSize: 00010000 indatSize: 000103d0 feaList addr: a408a0d8
3  feaSize before: 00010000
4  NTEA size: 00010fe8 : Buffer destination size
5  NTEA addr: 85905008 : Buffer start
6
7  #copy operations start here.
8  NEXT: FEA: a408a0dc NTEA: 85905008
9  MOV2: dst: 85905011 src: a408a0e1 size: 00000000
10 NEXT: FEA: a408a0e1 NTEA: 85905014
11 MOV2: dst: 8590501d src: a408a0e6 size: 00000000
12
13 ...
14 ...
15 ...
16 NEXT: FEA: a408aca8 NTEA: 85906c58
17 MOV2: dst: 85906c61 src: a408acad size: 00000000 #605 copies of 0 size
18 NEXT: FEA: a408acad NTEA: 85906c64
19 MOV2: dst: 85906c6d src: a408acb2 size: 0000f383 #606 copy
20 NEXT: FEA: a409a035 NTEA: 85915ff0
21 MOV2: dst: 85915ff9 src: a409a03a size: 000000a8 #607 copy
22 Breakpoint 6 hit
23 srv!SrvOs2FeaListToNt+0xd5:
24 97f7b63a be0d0000c0 mov esi,0C000000Dh #after the last copy, the next FEA is invalid
25 1: kd> !pool 85915ff9
26 Pool page 85915ff9 region is Nonpaged pool
27 *85905000 : large page allocation, tag is LSdb, size is 0x11000 bytes
28 Pooltag LSdb : SMB1 data buffer, Binary : srv.sys
29 1: kd> !pool 85915ff9+a8
30 Pool page 859160a1 region is Nonpaged pool
31 *85916000 : large page allocation, tag is LSbf, size is 0x11000 bytes
32 Pooltag LSbf : SMB1 buffer descriptor or srvnet allocation, Binary : srvnet.sys
33 1: kd> !pool 85915ff9+a5
34 Pool page 8591609e region is Nonpaged pool
35 *85916000 : large page allocation, tag is LSbf, size is 0x11000 bytes
36 Pooltag LSbf : SMB1 buffer descriptor or srvnet allocation, Binary : srvnet.sys
37 1: kd> dd 85915ff9 85915ff9+a8
38 85915ff9 00000000 00000000 00000000 ff000000
39 85916009 000000ff ff000000 000000ff 00000000
40 85916019 00000000 00000000 00000000 00000000
41 85916029 00ffdf11 00000000 20000000 00ffdf10
42 85916039 fffffd11 60ffffff 00100400 80000000
43 85916049 00ffdfef 10000000 fffffd00 18ffffff
44 85916059 fffffd01 00ffffff 00000000 00000000
45 85916069 00000000 60000000 00100400 00000000
46 85916079 00000000 90000000 fffffc1f 00ffffff
47 85916089 00000000 80000000 00000010 00000000
48 85916099 00000000 33000000 00000019
49

```

Figure 4: Stack trace of the copy operation

Once the stack trace ends and the program hits the breakpoint we set at "srv!SrvOs2FeaListToNt+0xd5", we have all the information we need to analyze the buffer overflow. There is a string of empty FEA structures at the start of the data section, and 605 zero-sized copy operations are performed. The size of the 606th FEA copied is F383 (62339), and the copy operation ends at "85915ff0".

After the 606th copy operation, the end of the buffer is reached at $85905008 + 10FE8 = 85915FF0$, at which point the loop begins again, this time with size A8 (168) and the next memory area, the pool in SRVNET.sys, is overwritten.

The 607th FEA copied is corrupted, so the server returns STATUS_INVALID_PARAMETER (0xC000000D), the last FEA of the NT Transaction sent to the server.

3.193.148	SMB	1287 Trans2 Secondary Request [Malformed Packet]
3.193.1	TCP	54 445 → 6022 [ACK] Seq=528 Ack=68016 win=65536 Len=0
3.193.1	SMB	146 Trans2 Response<unknown>, Error: STATUS_INVALID_PARAMETER
3.193.148	TCP	54 6022 → 445 [ACK] Seq=68016 Ack=620 win=524800 Len=0
3.193.148	SMB	93 Tree Disconnect Request

0000	00 0c 29 95 98 26 00 50	56 c0 00 01 08 00 45 00	..)..&.P V.....E.
0010	04 f9 4a b2 40 00 80 06	a7 65 c0 a8 c1 01 c0 a8	..J.@... .e.....
0020	c1 94 17 86 01 bd e4 44	f4 26 2b 60 ed 54 50 18	D .&+.TP.
0030	08 03 14 4e 00 00 47 58	6a 66 31 48 6e 37 6d 6b	...N..GX jf1Hn7mk
0040	66 76 4a 79 6f 4b 6e 47	61 79 54 37 63 70 43 36	fvjyoKng ayT7cpC6
0050	79 4d 6f 48 6c 61 67 78	43 66 2f 37 50 55 4f 72	yMoH1agx Cf/7PUOr
0060	55 44 58 34 76 76 36 35	41 69 69 58 67 65 71 73	UDX4vv65 AiiXgeqs
0070	39 42 2b 6e 4d 53 2f 43	65 62 76 56 35 72 46 43	9B+nMS/C ebvv5rFC
0080	65 55 68 2b 63 37 50 6d	2f 2f 5a 56 59 49 65 45	eUh+c7Pm //ZVYIeE
0090	6f 32 4e 78 80 00 a8 00	00 00 00 00 00 00 00 00	o2NX.....
00a0	00 00 00 00 00 00 00 00	ff ff 00 00 00 00 00 00	
00b0	ff ff 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
00c0	00 00 00 00 00 00 00 00	00 f1 df ff 00 00 00 00	
00d0	00 00 00 00 20 f0 df ff	00 f1 df ff ff ff ff ff	
00e0	60 00 04 10 00 00 00 00	80 ef df ff 00 00 00 00	
00f0	10 00 d0 ff ff ff ff ff	18 01 d0 ff ff ff ff ff	
0100	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	
0110	60 00 04 10 00 00 00 00	00 00 00 00 00 00 00 00	
0120	90 ff cf ff ff ff ff ff	00 00 00 00 00 00 00 00	

Figure 5: Broken FEA and server response

The EternalBlue exploit

overflows in non-paged pool memory, or more precisely, in large non-paged pool memory, which does not have a pool header and so reserves memory for use by other drivers after the non-paged pool is exhausted.

Therefore, the exploit is achieved by manipulating the next memory area of the overflowed buffer. In other words, EternalBlue manipulates the buffer of the SRVNET driver. However, both buffers must be aligned in memory. To align the non-paged pool, EternalBlue places data in the kernel pool as follows:

1. Create multiple SRVNET buffers (allocate pool space)
2. Free some buffers to create unused space to copy the SRV buffers into
3. Sending an SRV buffer to overflow the SRVNET buffer

■Mechanism of vulnerability attackThe

vulnerable code that causes a buffer overflow runs in the non-paged memory area of kernel memory. It also runs in the large non-paged area. These non-paged pools do not have a pool header at the beginning of the paged area, so a special technique is used in the EternalBlue vulnerability attack that exploits them. In this case, it is necessary to rewrite the header section reserved in the overflow area, as shown in Figure 6.

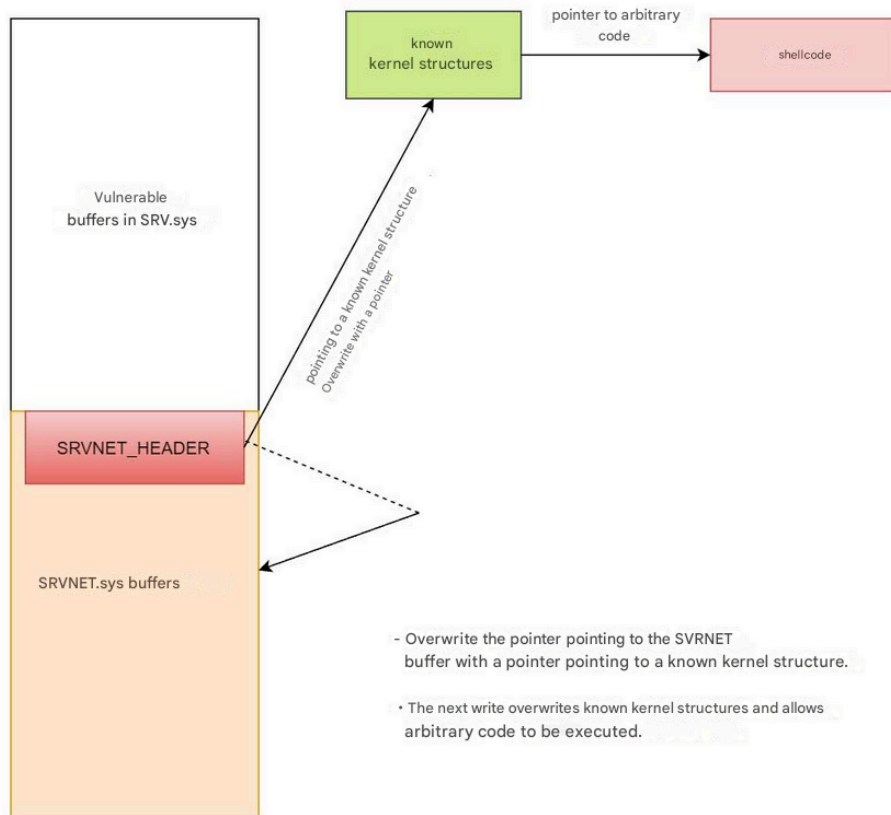


Figure 6: EternalBlue vulnerability attack mechanism

The idea behind this vulnerability is that multiple SRVNET buffers are created in memory (known as "kernel grooming"). This explanation intentionally omits other details to prevent malicious use of the analysis results.

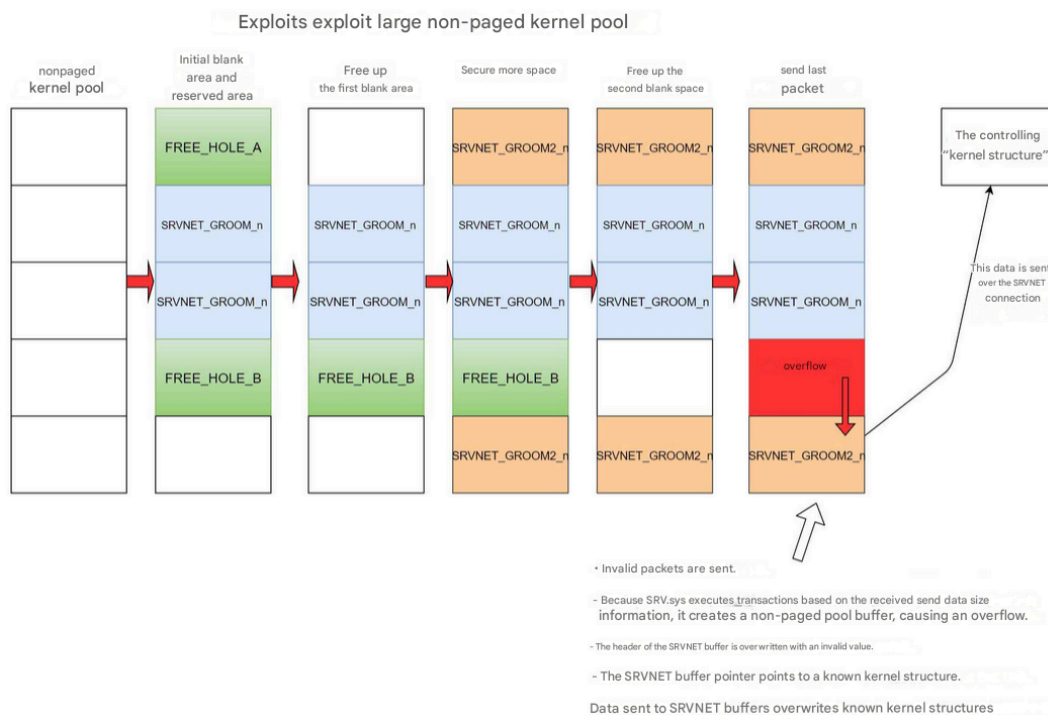


Figure 7: EternalBlue exploit vectors

■EternalBlue vulnerability attack route

EternalBlue's vulnerability attack exploits vulnerabilities in PCs and networks in the steps shown in Figure 7.

The exploit first sends the SRV buffer except for the last packet so that a large non-paged pool buffer will be created when the last data of the transaction arrives at the server. The SMB server accumulates data in the input buffer until it has read all the transaction information, which is specified in the first TRANS packet. After this is complete, the SMB server starts to process the data. In this case, the data is read via the Common Internet File System (CFIS) and passed to SrvOpen2.

At this point, the server confirms that all sent data has been received and sent in an SMB ECHO packet. This ECHO command is important if the victim's network is slow.

In this analysis, at the time of the initial data transmission, the "vulnerable buffer" has not yet been created in memory. Kernel grooming is then performed to allocate a vulnerable SRV buffer immediately before the SRVNET buffer. This kernel grooming involves the following steps:

1. FreeHole_A: Send an SMBv1 packet to initiate the creation of kernel hole A
2. SMBv2_1n: Send SMBv2 packets
3. FreeHole_B: Send another buffer to create a new kernel hole (sent before freeing the previous hole to create another hole)

4. FreeHole_A_CLOSE: Close the connection to free the buffer, then close A to create unused space.
5. SMBv2_2n: Send SMBv2 packets
6. FreeHole_B_CLOSE: Close the connection to free the buffer.
7. FINAL_Vulnerable_Buffer: Send the last packet of the vulnerable buffer

This creates a vulnerable buffer in memory just before the SRVNET buffer and parts of SRVNET are overwritten. And since the last FEA is corrupted, the conversion from FEA list to NTFEA list returns an error. In this case, the server returns STATUS_INVALID_PARAMETER (0xC000000D).

■How to avoid becoming a victim

Considering that EternalBlue was the entry point for many pieces of malware that caused serious damage to individuals and businesses around the world, it is clear how important it is to apply patches and keep your PC and network up to date. [Patches for Windows](#) against EternalBlue have already been released, including for [operating systems that are no longer supported](#).

In addition to regular patch management for PCs and networks, other recommended measures include the use of intrusion detection and prevention systems, disabling obsolete protocols and ports (e.g. 445), monitoring network traffic, protecting endpoints, [categorizing information](#) to mitigate damage, and [segmenting networks](#). [Virtual patching](#) is also effective against unknown vulnerabilities.

■**Trend Micro** customers who use [Trend Micro Deep Security™](#), a comprehensive security product for servers, [are protected from threats that exploit these vulnerabilities. In addition, the sandbox and file analysis engines in our network behavior monitoring solution, the Deep Discovery™ family](#), are able to detect this threat by its behavior, even without other engine or pattern updates.

For details on Trend Micro's countermeasures against EternalBlue, please see the technical support page below.

- <https://success.trendmicro.com/solution/1117192>
- <https://success.trendmicro.com/solution/1117391>
- **Reference articles:**